

Browser-Based Multiplayer RPG (OGame-Inspired) – Technical Specification and Development Plan

Introduction and Game Overview

OGame is a classic browser-based space strategy MMO where each player develops a space empire over time. Inspired by OGame's real-time, persistent universe, the proposed game will feature empire building across planets, technology research, and strategic combat in a multiplayer setting. Players start with a single planet and gradually expand by constructing buildings, mining resources, researching new technologies, and building fleets of ships. They can form alliances with others, engage in trade, and launch coordinated attacks on rivals – all in a continuous real-time universe that progresses even when players are offline. Success will require **careful resource management, planning, and strategy**, much like the original OGame.

Goal: Design and implement a full-stack **browser-based multiplayer RPG** (strategy MMO) with real-time gameplay and all core mechanics of OGame, including planet management, resource production, fleet warfare, tech trees, alliances, rankings, and monetization features. The entire stack will use JavaScript/TypeScript – a Node.js backend and an HTML5/Canvas front-end – to allow a seamless web experience without plugins. Users will even be able to customize the game's look by applying their own CSS themes to the UI.

Below is a **comprehensive technical specification** covering game features and the system architecture, followed by a development roadmap.

Game Mechanics and Features

Real-Time Multiplayer Gameplay

- **Persistent Real-Time Universe:** The game runs in real time on the server – meaning actions like resource gathering, building construction, and fleet travel happen over fixed durations even when players are offline. All players share a continuous universe (or multiple “universe” servers) that updates in real-time.
- **Concurrency and Low Latency:** Players can interact simultaneously. The server will push updates to clients instantly using WebSockets, ensuring low-latency bi-directional communication. WebSockets are ideal for real-time multiplayer games in browsers, offering *persistent* connections and minimal overhead. We will use a WebSocket library (e.g. **Socket.io** on Node.js) to handle real-time messaging. Socket.io simplifies the WebSocket handshake and provides convenient pub/sub messaging channels for game events.
- **Authoritative Server:** To prevent desync or cheating, the server will be authoritative for game state. Clients send input (e.g. commands to build or attack) and the server validates and applies game rules before broadcasting outcomes. This prevents clients from manipulating game logic

illegitimately. All critical computations (combat resolution, resource updates, etc.) happen on the server, with clients only presenting the results.

- **Game Loop / Tick Rate:** A game loop on the server will handle timed events and state updates. Instead of a single global loop that ticks for all players, the design will schedule events and use targeted updates:
- Many actions are event-driven (e.g. a building completes at a timestamp, or a fleet arrives at a certain time). The server can maintain a schedule of these future events and process them when their time comes (using timers or a priority queue of events).
- A periodic tick (e.g. every second) may run for continuous updates that require broadcasting (for example, update resource accumulation or send countdowns). These ticks will be optimized to broadcast only relevant data to each player (or each group of players) rather than everything to everyone. For instance, Socket.io “rooms” will be used to group players by context (like players viewing a particular battle or those online in a certain galaxy sector) so that tick updates can be emitted only to interested clients.
- Node.js being single-threaded means each game server instance handles one loop of events – we will ensure the loop is efficient and non-blocking. Heavy computations (like a large combat simulation) will be broken into smaller tasks or offloaded to worker threads if needed to keep the server responsive.

Planet and Fleet Management

- **Planet Colonization and Management:** Each player begins with a home planet. Planets serve as bases where players construct buildings and extract resources. The universe is divided into coordinates (galaxies, systems, planet slots) similar to OGame for expansion. Players can **colonize new planets** by building colony ships and sending them to free planet slots. Planet data (coordinates, owner, size, etc.) will be stored in the database and associated with the owning player’s account.
- **Building Infrastructure:** On each planet, players can build and upgrade a fixed set of structures (mines, power plants, factories, labs, defenses, etc.). Unlike some RTS games, each structure can be built only once per planet and then upgraded to higher levels. The **tech tree and building prerequisites** will be defined in a configuration (e.g. JSON) that the server references to enforce rules (for example, a *Shipyards* might require a certain level of *Robotics Factory*). Construction takes time – when a player queues a building upgrade, the server records a *finish timestamp* for that construction. A scheduled event or tick will complete the build at that time, updating the building level in the database and notifying the player.
- **Fleet Management:** Players can construct ships (e.g. fighters, cargo ships, cruisers, etc.) at shipyard facilities and organize them into fleets. A **fleet** is a group of ships assigned to a mission. The game will provide a Fleet Management UI for dispatching missions:
- **Fleet Missions:** Supported missions will include *Attack*, *Transport*, *Deploy* (relocate fleet), *Espionage*, *Colonize*, *Harvest Debris*, etc., mirroring OGame’s options. The player will select ships and a target, and the server will calculate mission details (travel time based on ship speeds and distance, fuel cost from **Deuterium** consumption, payload capacity, etc.).
- **Travel and Timing:** When a fleet is sent, the server creates a fleet object with its origin, destination, mission type, composition of ships, and timing (departure time = now, arrival time = now + travelDuration). This can be stored in-memory and in the database. The fleet is effectively “in flight” and not present at origin or destination until arrival. *En route* fleets are safe from attack (as in OGame, where fleets in transit can’t be intercepted mid-flight).

- **Arrival and Resolution:** When the arrival time is reached, the server triggers an event to resolve the mission. For an Attack mission, this means initiating **combat simulation** between the attacking fleet and the defender's forces at the target. For a Transport, the resources would be unloaded at destination, etc. After resolution, the fleet may return (e.g. survivors of combat or just completing delivery).
- **Fleet Saving:** Advanced players can "fleet-save" (send their fleets on dummy missions to avoid being destroyed while they're offline). Our design will incorporate all such tactics by supporting flexible mission scheduling and allowing recall of fleets mid-mission.
- **Fleet Fuel and Capacity:** Every ship has a fuel consumption (deuterium cost) for travel and a cargo capacity. The server will calculate fuel needed for the trip and deduct it from the origin planet's deuterium. Cargo transfer (resources moved on transport missions or loot from raids) will be limited by ship capacity, which is computed as sum of cargo holds minus fuel usage.
- **User Interface for Fleets:** The front-end will show a **fleet dispatch screen** (to choose mission and ships) and a **fleet movement overview** listing ongoing missions and time to arrival (with dynamic countdowns). This UI will be updated via real-time events – e.g., the server might emit an update when a fleet status changes or a countdown tick every second for consistency.

Resource Gathering and Economy

- **Resource Types:** The core resources are **Metal, Crystal, and Deuterium** (matching OGame's economy). Metal and Crystal are produced by mines on each planet, while Deuterium is produced by synthesizers (and consumption for fuel). Additionally, *Energy* is a secondary resource that powers mines (produced by solar plants, fusion reactors, etc.). We will also implement a **premium currency** (e.g. "Dark Matter") used for special purchases in the shop, analogous to OGame's Dark Matter.
- **Resource Production:** Each resource-producing building (mines, synthesizer) generates a certain amount per hour. The production rate is determined by building level and possibly technology bonuses. Instead of updating resource counts every second, the server will use a **lazy evaluation** strategy for efficiency:
- Each planet's record stores its last resource update timestamp and current stored amounts. When a player views their planet or tries to spend resources, the server calculates how much has accrued since the last update (based on rate and elapsed time) and updates the stored amount ¹. This avoids needing a constant tick for every planet.
- Optionally, a periodic process (e.g. every few minutes) can batch-update all planets' resources in the database, but on-demand calculation will likely suffice and scale better.
- **Resource Storage and Cap:** Storage buildings determine the cap for each resource. The server will enforce that production beyond storage capacity is not accumulated (or at least not beyond some buffer). This will be part of the game logic checks on resource update.
- **Upgrading & Costs:** Building upgrades, ship construction, and research all consume resources. The costs and formulas (often exponential growth per level) will be defined in the game data config. The server deducts resources *at the start* of a construction/research action to prevent duping, and the action then completes after the time delay.
- **Trading:** Players can trade resources (either directly or via a marketplace system). This could be implemented via a simple **trade offer** listing (like a marketplace where players post offers and others accept) or direct *sending resources* by fleet transport missions. In either case, the server will

handle the secure exchange: e.g., if using a marketplace, when an offer is accepted both players' resources are adjusted in a single atomic operation.

Building Construction and Technology Trees

- **Building System:** Each planet has a set of buildable structures (e.g. Metal Mine, Crystal Mine, Deuterium Synthesizer, Solar Plant, Fusion Reactor, Robotics Factory, Shipyard, Research Lab, Nanite Factory, Storage Silos, etc.). Buildings unlock capabilities or improve output (mines increase resource hourly yield, labs enable research, shipyard enables ship building, etc.). The **construction interface** will list available buildings and their next level cost/time.
- **Construction Timers:** Only one building can be constructed per planet at a time (as in OGame). When a construction is started, the server creates a build job with a finish timestamp = now + buildDuration. The planet is "busy" until then. The client will show a countdown. On completion, the server will increment the building level and free the build queue. This completion event will trigger a WebSocket message to the player so their UI can update (and possibly a notification if offline).
- **Technology Research:** The game will implement a tech tree for research (e.g. Energy Tech, Laser Tech, Armor, Weapons, Shielding, Espionage, Computer tech, Astrophysics for colonization slots, etc.). Research is conducted at the Research Lab on planets and similarly takes time. Only one research can be active at a time (globally or per lab – likely globally per account as in OGame). Research often has prerequisites (both building levels and other research). We will store a directed acyclic graph of tech prerequisites and check it before starting a new research.
- **Tech Tree Data:** All buildings and researches and their dependency tree will be defined likely in a JSON file or database table. For example, to research *Intergalactic Research Network* might require multiple labs across planets, etc. The **game config** will outline costs (often increasing exponentially or by formula each level). This config-driven approach makes balancing easier via the admin panel (an admin could tweak constants in this config for game balance without code changes).
- **Construction and Research Speed-Ups:** We may introduce mechanics like *Robotics Factory/Nanite Factory* that reduce building construction time, or *Research Labs* that reduce research time (or linking labs via tech, like OGame's Intergalactic Research Network). These effects will be implemented in the time calculations on the server.
- **UI and Customization:** The building and research pages on the front-end will likely be HTML/CSS with dynamic values (not pure canvas, since these are mostly text/info displays). We will allow players to apply their own CSS to these pages. For example, a user can supply a custom stylesheet that overrides the default theme (colors, layout) for the game UI. The application will load the user's CSS after the default styles so it can override them. We will need to sanitize or limit this CSS to prevent abuse (e.g. disallowing any CSS that could obscure important UI or phish, and ensuring it only affects their view) – possibly by scoping user CSS to specific classes or using a whitelist of allowed style properties.

Combat Simulation

- **Battle Initiation:** Combat occurs when an attacking fleet arrives at a target planet (or moon). At that moment, if the target has defending units (ships or defensive structures), a battle is simulated. If the planet is undefended, the attacker wins automatically and can loot resources.
- **Round-Based Simulation:** The combat system will follow OGame's mechanics closely for authenticity. Each battle runs for up to **6 rounds** maximum. In each round:

- All surviving attacking and defending units engage. Each unit targets a random enemy unit and fires based on its weapon power, shield, and hull values (which are modified by the player's technology levels) ² .
- The damage calculations will incorporate the rules: If a weapon's strength is below a small threshold of the target's shield, the shot bounces off harmlessly; otherwise shields absorb damage until depleted, and any excess damages the hull. If a unit's hull falls below 70% it might **explode** (get destroyed) with a probability proportional to damage.
- Some ships have **rapid fire** against certain ship types, meaning they may get extra shots if they destroy a target. All these rules will be coded into the combat resolution module.
- After each round, destroyed units are removed, shields on surviving ships regenerate to full for the next round, and hull remains at whatever value if the ship survived.
- **Battle Outcome:** The battle ends immediately if one side loses all units. If both sides still have forces after 6 rounds, the battle is a *draw*. Possible outcomes:
 - **Attacker wins:** All defender ships/defenses destroyed (if attacker wins or if defender had none to start). Attacker can then loot a portion of the resources from the planet.
 - **Defender wins:** All attacking ships destroyed (defender's remaining units hold the planet; no loot for attacker).
 - **Draw:** Combat stops after 6 rounds with both sides having survivors; in this case, the attacker fails to loot (same as defender win in terms of loot outcome) and the surviving fleets simply return home.
- **Post-Combat and Debris:** When ships are destroyed, we will create a **debris field** at the location (space debris composed of a percentage of the destroyed ships' metal and crystal). Players can send Recycler ships on a Harvest mission to collect this debris. The game will track debris fields per location in the database.
- **Rebuilding and Repairs:** In OGame, defensive structures have a chance (e.g. 70%) to repair (be rebuilt for free) if destroyed in combat. We will implement a similar mechanic for defenses to make combat more balanced.
- **Combat Simulation Implementation:** This will be a **pure server-side module** (to prevent any tampering). When combat is triggered, the server will:
 - Gather the attacker's fleet stats and the defender's fleet/defense stats (including all tech upgrades).
 - Simulate round by round as per rules above.
 - Determine outcome and calculate losses, debris, and loot.
 - Update the database: remove lost ships from respective owners, remove or damage defenses on planet (with chance of repair), create debris record, calculate and credit loot to attacker (attacker's ships can carry back at most their cargo capacity).
 - Send results to the involved players (and possibly to an alliance chat or combat report system).
- **Battle Reports:** The server will generate a battle report (summarizing rounds, losses, loot) that is saved and delivered to the players via in-game messages. This is important for players to review outcomes. It also aids in debugging the combat algorithm during development.

Alliance System

- **Alliance Formation:** Players can create or join Alliances (guilds). An alliance is essentially a group with its own name, tag, member list, and leadership hierarchy (e.g. one founder/leader, optional co-leaders, and members). The server will provide endpoints to create an alliance (with a unique name/tag) and join or leave alliances. Alliance data is stored in the database (alliance collection with references to member player IDs and roles).
- **Alliance Communication:** Alliances will have private communication channels:

- **Alliance Chat:** A real-time chat channel only visible to members. This can be implemented via a Socket.io room for the alliance, or via an in-game messaging board.
- **Alliance Circular Messages:** The ability for leaders to send a message to all members (we can implement this as a special in-game message type).
- **Cooperative Gameplay:** Alliances enable cooperative strategies:
- **ACS (Alliance Combat System):** Multiple alliance members can coordinate attacks or defenses on the same target. If enabled on a server, alliance members can send fleets to join an ongoing attack or to defend an ally's planet ³. Our combat system will support multiple attackers and defenders: essentially, if fleets from different players arrive at a planet at the exact same time for an attack/defense mission, the server will treat them as one combined side in the battle. We'll need to handle the timing and grouping logic (possibly allowing players to schedule *ACS attacks* by inviting others and aligning arrival times).
- **Resource Sharing:** Allies might send resources to each other via transport missions. We might also implement an **Alliance Depot** functionality (as in OGame) where a player's Alliance Depot building allows stationing allied fleets in orbit and refueling them.
- **Alliance Management:** Features for alliance leaders:
 - Manage applications (if alliance is invite-only or application-based, review and accept new members).
 - Set alliance diplomacy statuses with other alliances (perhaps simple flags like allied, at war, neutral).
 - Disband alliance, remove members, promote/demote members.
 - An **alliance ranking** can be derived (sum of members' scores).
- **Alliance UI:** A dedicated alliance page will show alliance info, member list, their scores, and online status possibly. Also links to alliance chat or forum if implemented. Users can customize their alliance with a description or logo – we'll allow HTML-safe formatting or preset templates to avoid XSS issues.

Player Accounts and Authentication

- **Registration & Login:** The backend will expose RESTful APIs (via Express) for user registration and login. Upon registration, a new player account is created with initial settings (starting planet, default resources, etc.). Passwords will be stored securely (hashed with a strong algorithm, e.g. bcrypt). Login will issue an authentication token (likely a JWT if using a token-based stateless auth, or a session cookie if using session-based auth).
- **Authentication Method:** We can use **JWT (JSON Web Tokens)** for a modern approach: the client gets a token on login and then uses it for authenticated WebSocket and HTTP requests. Socket.io can be configured to accept a token for handshake authentication. Alternatively, we use Express sessions stored on the server (with a session ID cookie) – if so, use a secure session store (e.g. Redis-based) to allow scaling beyond one server process.
- **Account Management:** Players should be able to manage basic profile settings (email, password change, etc.). Email verification can be considered for security (optional in initial version).
- **Multi-Universe Support:** If multiple game universes are offered (like different servers with different rates or rules), the account system will allow choosing a universe or having separate accounts per universe. For simplicity, we might treat each universe as a separate deployment or separate database schema.
- **Security:** Protect against common web vulnerabilities in auth:
 - Brute-force protection (rate limiting login attempts, CAPTCHA after many failures).
 - Use HTTPS for all communications (to safeguard credentials and tokens).
- **Input validation** on all user data (login forms, etc.) to prevent injection attacks or malformed input.

In-Game Shop and Monetization

- **Premium Currency:** The game will include a premium currency (like OGame's *Dark Matter*) that players can purchase with real money. This currency can be used to buy convenience items or bonuses in the in-game shop. We will not make it pay-to-win, but typical purchases could be: time boosters (speed up a construction or research), resource packs, or cosmetic enhancements. Premium currency balance is stored on the player's account and must be handled securely to prevent tampering.
- **In-Game Shop:** A storefront UI will list available purchasable items/packs. For example:
 - Resource bundles (e.g. +100k Metal).
 - Temporary boosts (e.g. +20% mine production for 7 days).
 - Officers or special roles (as in OGame: Commanders, Officers that give various account perks).
 - Skins or cosmetic themes (if we implement cosmetic customization beyond user CSS).
- **Payment Integration:** For real money transactions, we will integrate a payment gateway (such as Stripe or PayPal). The Node.js backend (in a secure route) will handle payment webhooks or purchase callbacks to credit the premium currency. All real-money transactions will be done over HTTPS and follow best practices (e.g. verifying signatures from the payment provider).
- **Anti-Fraud:** We will implement basic anti-fraud checks – for instance, wait for payment confirmation from gateway before crediting currency, log transactions, and perhaps limit purchase amounts or require additional verification for large purchases.
- **Ads Integration:** Aside from direct purchases, the game will generate revenue via advertisements:
 - We will designate certain UI areas for banner or video ads (e.g. a banner in the sidebar or a pop-up video ad for optional reward).
 - Use a web ad network (such as Google AdSense or a gaming ad network) by embedding their ad scripts in the frontend. The ad content will load in iframes or containers so it doesn't interfere with game code.
 - Ensure ad loading is asynchronous to not degrade game performance. We might only show ads outside of critical game screens (or offer an ad-free purchase option with the premium currency).
 - We will also consider **rewarded ads**: e.g. "watch an ad to get a small amount of Dark Matter or a resource boost," which can be implemented by triggering a video ad and then crediting a reward via a callback.
- **Monetization Balance:** All monetization features will be configured in such a way that they do not break the game's balance. The admin panel will allow tuning prices, rewards, and ad frequency.

Leaderboards and Ranking System

- **Player Ranking:** The game will calculate a score for each player based on their assets (e.g. points for each building level, each research level, and each ship/defense unit). This follows OGame's scoring where e.g. 1 point = 1000 resource spent, or a similar metric. The server will maintain a **leaderboard** of players sorted by score.
- **Leaderboard Implementation:** For efficiency, we can maintain an updated ranking in real-time:
 - Each time a player finishes a construction, research or builds/destroys units, their score is recalculated or incremented accordingly in the database.
 - We can use a **sorted set in Redis** to keep track of top scores, as Redis can update and rank scores very quickly in memory ⁴ ¹. Alternatively, we can periodically recompute rankings from the main database (but that is heavier; a hybrid approach is best: update incrementally, and periodically verify).

- The top N players' standings will be exposed via an API or cached for the frontend to display. We'll provide leaderboards for overall ranking, as well as sub-rankings (e.g. by category: most powerful fleets, top researchers, etc., if desired).
- **Alliance Ranking:** Similarly, an alliance ranking can be maintained by summing member scores. Whenever a member's score changes, we update their alliance's total score.
- **Leaderboard UI:** A Rankings page will show the list of players (and alliances) sorted by rank, with pagination. It will show each player's name, score, and perhaps alliance. The data on this page can be served via a REST API request (it doesn't need to update every second, possibly can be refreshed on demand or every few minutes). A caching layer (in-memory or CDN) can be used to handle frequent requests for the leaderboard without hitting the DB constantly.
- **Historical Data:** We might store snapshots of scores over time for analytical or "hall of fame" purposes, but that's an optional feature. At minimum, the current rankings are maintained.

Administration and Moderation Tools

- **Admin Panel:** A secure admin web interface (protected by additional login or IP whitelist) will be developed for game masters to monitor and manage the game. This will likely be a separate section of the web app (e.g. accessible only to users with an admin flag). Key features of the admin panel:
- **Player Management:** Search for players by name or ID, view their account details (planets, resources, fleets, researches, last login, IP if needed for multi-account detection). Admins can perform actions: adjust resources (for event rewards or testing), force fleet recalls, teleport players, or **ban/suspend** accounts. Banning might set a flag that prevents login and optionally puts their planets in vacation mode.
- **Game Balance Settings:** Modify global parameters such as resource production multipliers, building and research speed, combat parameters, etc. These settings could be stored in a config file or database table that the game uses. The admin UI will provide controls to change these values (e.g. change universe speed from 1x to 2x) and the changes can propagate to the server (possibly requiring a restart or being coded to take effect live depending on implementation complexity).
- **Moderation Tools:** View in-game messages or chat logs for moderation (to investigate harassment or cheating discussions). Admins should be able to delete offensive messages or mute players if necessary. This requires storing chat logs or at least recent messages – possibly we'll log chat to a database with timestamps.
- **Game State Monitoring:** Dashboard of current server state – number of online players, server performance metrics (CPU, memory), maybe queue lengths. This helps admin see if the game is running smoothly.
- **Impersonation:** (Optional) The ability for an admin to impersonate or log in as a player to debug an issue.
- **Universe Management:** If multiple universes, tools to create new universe instances or merge universes (advanced).
- **Admin Security:** The admin panel will have strong security:
 - Admin accounts will have very strong password requirements and possibly 2FA.
 - All admin actions should be logged (audit log) – e.g. if an admin gives themselves resources or bans someone, it's recorded.
 - The admin panel will be served only over HTTPS and maybe on a separate subdomain or path. We might also restrict access by IP or require VPN for admins, depending on the deployment context.
- **Anti-Cheat/Monitoring:** Apart from reactive moderation, we'll include automated checks:
- **Multi-account detection:** Flag if the same IP or device is being used for multiple accounts often (could indicate a player controlling multiple accounts, which might be against rules).

- **Bot detection:** Monitor unusual patterns (e.g. actions being performed 24/7 or with superhuman consistency). We can track metrics like clicks per minute, or use honeypot timing (like detect if a human reaction time is too perfect). While hard to fully prevent, we can flag suspicious accounts for review.
- **Rate limiting:** The server API will include rate limits on certain actions to mitigate bots or spammers. For example, limit how often a player can probe another, or send messages, etc., beyond normal gameplay limits.
- **Data validation:** As mentioned, the authoritative server approach ensures that any critical action from clients is verified. For example, if a client claims “build X completed,” the server will *ignore* that and rely on its own timing. This prevents tampering via client-side hacks.
- **Game Balance Adjustments:** Using the admin interface or config, the live game variables (like production rates, combat damage ratios, etc.) can be tweaked to keep the game fair and fun. For example, if a certain strategy is overpowering, the admin can nerf a particular ship or tech. These changes could be applied between rounds or with server restarts as needed.

Technical Architecture and Stack

Overview of System Architecture

The application will follow a **client-server model** with a clear separation of concerns: - The **Node.js server** (backend) contains all game logic, persistent data handling, and real-time communication. - The **browser client** (frontend) handles user input and presentation (rendering the UI, Canvas graphics, etc.), communicating with the server via HTTP and WebSockets.

Technology Stack Summary: - **Server:** Node.js (JavaScript/TypeScript). We will use the Express.js framework for HTTP API endpoints and static file serving. Express is a minimal, flexible Node framework that provides a robust set of features for web applications. On top of Express, we'll integrate Socket.io (or the `ws` WebSocket library) for real-time features. - **Database:** A combination of **NoSQL and in-memory store**. The primary game data will reside in a NoSQL database like **MongoDB** (document-oriented) to allow flexible schema evolution for complex game objects (players, planets, fleets). MongoDB's flexible JSON storage suits game data (which can be naturally represented as nested documents, e.g. a player document containing an array of planet sub-documents) and can handle a large volume of data with sharding if needed. For caching and fast transient data, we'll use **Redis**, an in-memory data store known for sub-millisecond latency ⁴. Redis will be used for things like session storage, leaderboards, and pub/sub messaging between server instances ¹. This hybrid approach leverages each DB's strengths (MongoDB for persistence and flexible queries, Redis for speed and real-time tasks ¹). - **Frontend:** HTML5, CSS, JavaScript in the browser. The game interface will be largely web-based (standard DOM for forms, menus, and info, which allows user-supplied CSS to skin it). For any dynamic graphical components (like animations, star maps, battle visualizations), we will use an **HTML5 Canvas** or WebGL. We might leverage a Canvas drawing library or a small game engine (e.g. **Pixi.js** for 2D rendering or even **Phaser** if a lot of game-like rendering is needed). However, since OGame's style is mostly menu-based with relatively simple visuals, we can manage with custom Canvas code for specific features (like drawing graphs or visualizing combat) and rely on standard HTML/CSS for the general UI. - **Communication Protocols:** HTTP/HTTPS for initial page loads and REST API calls, and WebSockets (secure WSS) for live game state pushes and user commands. WebSockets will keep the client and server in sync in real time, which is crucial for a smooth multiplayer experience. We will ensure the WebSocket connection remains open for the duration of play, falling back to reconnection or alternative methods if needed (Socket.io handles reconnection logic and can fall back to

HTTP long-polling automatically if WebSocket fails). - **Server Hosting:** The Node.js server can be containerized (Docker) and hosted on cloud VM or container platforms. We plan to design the server to be **stateless** aside from the database – meaning multiple instances can run behind a load balancer to scale to many concurrent users. Sticky sessions or a shared session store (Redis) will be used so that WebSocket connections can be balanced properly (Socket.io with Redis adapter will allow all server instances to broadcast to the correct clients across nodes).

The high-level architecture involves: **(Client) ↔ (Node.js + Express + Socket.io Server) ↔ (MongoDB + Redis)**. All game modules (combat, economy, etc.) run server-side within the Node server process.

Backend: Server-Side Implementation

Web Framework and API Design

We will use **Express.js** on Node as the core web framework. Express will handle: - Serving the main application pages and assets (HTML, CSS, client JS, images). The game's front-end might be a single-page application or a multi-page app with some dynamic loading – either way, Express can serve the necessary files (perhaps using a templating engine for server-side rendering of initial page, or just serving an index.html for an SPA). - Providing a set of RESTful **HTTP APIs** for certain game actions and data retrieval. For example: - `POST /api/register` – register a new user. - `POST /api/login` – authenticate and return token. - `GET /api/player/:id` – get player profile (if viewing others). - `GET /api/planets` – get current user's planets data. - `POST /api/build` – initiate a building construction. - etc. Many of these actions could also be done over WebSocket messages instead of HTTP POST for a more real-time feel (e.g. the client could emit a “build” event over socket). We will likely support both in some cases, but using the socket for in-game commands after login makes a truly live single-page experience. - **Express Middleware:** We will use middleware for common tasks: body parsing (JSON requests), cookie parsing (if using sessions), authentication checking (e.g. a JWT verification middleware on protected routes), and rate limiting (to mitigate abuse of APIs). - **Session Management:** If we opt for sessions, we'll use `express-session` with a Redis store. If using JWT, we'll verify tokens in a middleware for protected routes and also in the initial WebSocket handshake. Each approach has tradeoffs; JWT offers stateless scalability, whereas sessions are simpler to revoke. We might combine them (e.g. JWT for game actions and separate session for admin). - **MVC Structure:** Although not heavy, we'll structure the server code logically: - **Models:** representing game objects (Player, Planet, Fleet, etc.), possibly using an ODM like **Mongoose** if MongoDB, to define schemas and relationships. - **Controllers / Routes:** Express route handlers for each API, which call underlying services. - **Services/Logic:** Modules encapsulating game logic (e.g. a `CombatService` to run battles, `EconomyService` to compute resource flows, `AllianceService`, etc.). - This separation makes it easier to test logic in isolation and maintain the code.

Real-Time Communication (WebSockets)

Real-time features are crucial. We'll integrate **Socket.io** on the Node server for simplicity in managing WebSocket connections and events: - When the client connects (after login), it opens a WebSocket connection to the server (`io.connect()` in Socket.io). We authenticate this connection either by a token or session cookie. - We define various **events/channels**: - For instance, `joinPlanet` event – when a player opens a planet view, the client can join a Socket.io “room” for that planet to receive updates (like resource change ticks, or if the planet gets attacked). - `global` or `universe` room – all players might join a global room for announcements or server-wide tick events (though careful with scale). - Alliance chat

room – members join a room like `alliance_<ID>` to chat and see alliance-related notifications. - Combat-specific room – if a combat is happening and we want to push real-time battle info (optional, could also just send once final result). - The server will emit events to clients to inform them of state changes: - Ex: When a construction is done, emit `buildingFinished` to that player. - When a fleet arrives or combat occurs, emit `combatReport` or `fleetUpdate`. - Periodic events: possibly emit `resourceUpdate` every X seconds with new resource totals (though as mentioned, we might just update on demand to reduce spam). - Chat messages, alliance requests, etc. all via sockets. - **Scalability:** Socket.io can scale with multiple nodes by using a **Redis adapter**. We will configure Socket.io to use Redis pub/sub so that a message from one server instance can be forwarded to clients connected to any other instance ¹. This is important when we have many players on different server processes – all subscribe to the same Redis channels for rooms. - **WebSockets vs WebRTC:** For this type of game, standard WebSockets suffice (it's not peer-to-peer action, but server-mediated). We won't need WebRTC for networking (WebRTC could be used for peer-to-peer if we had heavy real-time media or wanted to offload server, but here authoritative server is needed for security). - **Fallbacks & Heartbeat:** We will implement heartbeat checks – if a client disconnects unexpectedly (network issue), the server will mark them offline after a timeout and possibly put their account into "vacation mode" if they remain offline long (if OGame has such a feature). Reconnecting will re-sync their state. Socket.io provides ping/pong heartbeats out of the box and will auto-reconnect clients; we'll make use of that to ensure resilience. - **Performance considerations:** We will be mindful to not send too frequent or too large messages. For instance, rather than sending every single resource tick per second, we might send a bulk update once a minute or update only when the player opens the page. High-frequency updates will be limited to what's needed (like a second-by-second countdown only for the last few seconds of a build, or for a real-time combat if visualized). Testing will help tune this.

Game Loop and Event Scheduling

The server's game loop isn't a classic fixed-step loop for physics (like in action games) but rather a scheduler for events in an MMO style: - We will maintain a priority queue or timeline of upcoming events (could be as simple as an array or a specialized structure sorted by time). Events include things like: - Building/research completion times - Fleet arrival times - Periodic resource generation checkpoints (if we choose to periodically add resources) - Alliance event triggers (like war declarations end, etc.) - Implementation approach: - On server start, load any pending events from the database (e.g. all fleets in transit, all builds in progress with their end times). - Use `setTimeout` or `setInterval` in Node to check the nearest event. For example, find the next event due, and set a timeout for that exact time to process it. After processing, look for the next. - Because many events could happen at the same second, we might instead have a tick every 1 second or 0.5 second that processes all events whose time $\leq$ now. This is simpler and ensures if multiple events coincide they all get handled. - Node.js timers are not perfectly precise under load, but on average within a few milliseconds which is fine for a game where seconds matter more than milliseconds. - **Handling events:** When an event triggers, the server calls the relevant logic: - For fleet arrival: run combat if needed or handle transport. - For build complete: increment building level. - For resource tick (if used): add resources to each planet (though as said, we may avoid per-tick resource events). - **Multi-threading considerations:** Node is single-threaded for JS execution; heavy computations (like a giant combat) could block the event loop. If we anticipate very large battles (hundreds of thousands of units), we might incorporate a mechanism to offload combat calculation to a worker thread or a separate service. Node's `worker_threads` or a job queue (like a compute service) could handle intensive tasks asynchronously. However, initially we expect manageable computational load (typical battles will be moderate in size). - **Accuracy and Persistence:** We must ensure that if the server crashes or restarts, the game events are not lost. That's why we store all important future events in the database: - E.g. each fleet mission in a `fleets`

collection with its arrive time and mission details. - Each building upgrade in a `construction` collection with finish time. If a restart happens, the server can recompute what's done (if a finish time has passed while offline, it will complete immediately on startup). - **Time Sync:** We will base everything on server time (usually synced with NTP). The client will rely on server-sent timestamps to display timers, to avoid drift. If needed, we'll send the server time to clients periodically so they can adjust their countdown displays.

Data Model and Database

Using **MongoDB** for primary data storage: - **Schema Design:** - *Players:* Each player document contains username, password hash, email, signup date, alliance membership, current score, etc. - *Planets:* Planets could be sub-documents of player or separate collection. We might do separate for flexibility (one-to-many relation), but for quick access of all a player's planets, embedding might also work since the number of planets per player is limited (OGame limits via Astrophysics tech). Possibly, we store planets in their own collection with a reference to owner player ID (and unique coordinates index). This allows querying by location (for galaxy view, etc.). - *Buildings/Research:* These can be fields in the Planet (levels per building) and fields in Player (levels per research). Techs are at player level (account-wide) in OGame. So Player document might have a sub-document for research levels. Planet document has building levels. - *Fleets:* Each in-flight fleet could be a document in a Fleets collection (with fields: originPlanet, destPlanet, ships composition as an embedded list, mission type, arrival time, return time if round-trip, ownerId, maybe targetOwnerId for combat reference). - *Stationary military:* Ships located at a planet when not on a mission could just be part of that planet's state (like a hangar count). Defenses are part of planet state too. - *Alliance:* an Alliance collection with alliance name, tag, member list (or references), and maybe a stored alliance chat (though chat can be transient). - *Messages/Reports:* A collection for messages (player-to-player messages, alliance announcements, combat reports). We may keep combat reports in messages or a separate collection due to potentially large content. Each message has recipients, content, timestamp, type. - *Leaderboard:* We might not need a separate collection if we calculate on the fly, but we could have a cached Score collection. However, since score = sum of various things, we can compute when needed. We will use Redis sorted set for quick top N queries as mentioned. - **Why MongoDB:** The flexible document model easily accommodates evolving game data (e.g. if we add new ship types or new resources, we can just add fields or arrays without a full migration). Also, many game clones have used MySQL; it works, but NoSQL often provides better horizontal scaling and less impedance for object-like data structures. - **Redis Use Cases:** - Caching frequently accessed data: e.g. player session data (so that each WebSocket message doesn't require a Mongo lookup for user). - Leaderboards: Using Redis sorted sets to store `<score, playerId>` allows getting top players in $O(\log N)$ and updating scores in $O(\log N)$, which is very fast for thousands or even millions of entries. - Pub/Sub: As mentioned for Socket.io scaling – the adapter uses Redis pub/sub to coordinate. - Rate limiting counters: We can use Redis to count actions for rate limit (since Redis atomic increment and expire are useful for sliding window rate limiting). - Transient game state: While main state is in Mongo, some ephemeral states could live only in Redis or memory – e.g. a temporary event reward multiplier could be stored in Redis and expire. - **Database Transactions and Consistency:** MongoDB by default is eventually consistent for separate documents, but we will use transactions (MongoDB supports multi-document ACID transactions in modern versions) for critical operations that involve multiple documents. For example, if a trade happens transferring resources between two players, or if a combat destroys ships of two players and moves resources around, we can use a transaction to update all relevant documents consistently in one go. - **Indexes:** We will add indexes to optimize key queries: - Players by username (for login), by score (for ranking if needed). - Planets by coordinates (for galaxy view queries). - Fleets by arrival time (to query next events easily, though we might keep them in memory). - etc. - **Scaling DB:** We anticipate a need to scale reads of game data (many simultaneous players querying galaxy or their

planets). Using a combination of caching (in-memory caching of semi-static data like universe galaxy info) and possibly read replicas for Mongo if needed. Mongo can be sharded by player or by galaxy to distribute load if the user base is huge. Redis can handle a high throughput for the real-time parts. - **Backup and Persistence:** We will have regular backups of MongoDB (for disaster recovery). As the game state is valuable, enabling point-in-time recovery or replica sets for high availability is important. Redis data (for sessions etc.) can be regenerated or less critical to persist (except maybe leaderboards which can be recomputed). - **Data Volume considerations:** Each player can have multiple planets each with many buildings, but overall data per player is not enormous (kilobytes). The biggest data volumes might be message logs or combat reports if we store a lot. We can implement pruning of old messages or archive them to secondary storage after some time to keep DB size manageable.

Security and Anti-Cheat Measures

Security is paramount for an online game – we need to protect both the server from attacks and the game from cheating: - **Authentication Security:** As noted, hashed passwords, HTTPS, and possibly email verification/2FA. Use industry-standard libraries to avoid mistakes (e.g. Passport.js for OAuth if we allow Google/Facebook logins, etc., though not required). - **Input Validation and Sanitization:** All inputs from users (either via REST or WebSocket events) will be validated. We will define schemas for expected data (e.g. using a JSON schema or manual checks). This prevents malicious input (like sending a string where a number is expected, or overly large values). It also helps prevent **NoSQL injection** or other injection attacks. For example, when using Mongo with user input, we'll ensure to disallow operators that could manipulate queries. - **XSS and Content Security:** The client side could be vulnerable to XSS if we display user-generated content (like alliance descriptions, player messages). We will sanitize any HTML or disallow raw HTML in such content. For chat messages, for instance, we'll escape HTML characters so no one can inject a `<script>`. We might later allow a limited subset (like bold text or links) through a markup system. - **User-Supplied CSS Safety:** Allowing custom CSS is a potential vector for abuse if not limited. Malicious CSS could attempt to obscure UI or even trick users. We will implement safeguards: - Possibly only allow users to choose from predefined themes or allow custom CSS but strip out disallowed rules. For example, avoid `position: fixed` or external URL references. There are open-source libraries or we can write a simple parser to filter CSS properties. - Alternatively, isolate user CSS in scope by applying it only under a specific root element that contains game UI, to minimize impact. - This is a feature we mention, but we'll roll it out carefully, perhaps in beta, after core features. - **Preventing Cheating: - Authoritative Actions:** The server never trusts the client's authority on game outcomes. A client cannot, for example, just tell the server "I now have X resources" or "I destroyed that fleet" – it must request actions and the server computes results. This fundamental design stops most cheating. - **Speed Hacks:** Since time and production are server-driven, players cannot speed up their progress except through legitimate game mechanics or monetization. We will ensure the client can't bypass build timers (the server won't complete it until the actual time). - **Resource injection:** All resource changes go through server logic. We may add sanity checks like ensuring no negative resources or huge jumps occur without cause (which could indicate an exploit). - **Multi-account and botting:** As discussed, detection of suspicious behavior is key. We might implement captchas or interactive checks if a player is performing hundreds of actions with no delay (which a normal human wouldn't). - **Communication Encryption:** Use wss (WebSocket Secure) and https so data packets can't be easily sniffed or modified in transit. This stops basic MITM cheating or seeing other players' data (though in our design each player only gets their own data anyway). - **Server Security Best Practices:** On the Node server, apply OWASP Node.js best practices: - Keep dependencies up to date to avoid known vulnerabilities. - Use helmet middleware for setting secure HTTP headers (CSP, HSTS, etc.). - Limit payload sizes to mitigate DoS by large payload. - Possibly run Node with limited privileges on the host and use container isolation.

Also, use a reverse proxy (nginx) to handle SSL and rate limiting at the network level. - Protect against common web vulnerabilities (SQLi is not in Mongo typically, but NoSQL injection as noted, XSS, CSRF for any state-changing HTTP forms if we have them – though a lot is via sockets and APIs where we include auth tokens). - For CSRF, since we'll likely do an SPA with JWT, it's less of an issue, but any forms should have CSRF tokens or same-site cookies. - **Anti-DDoS and Scalability:** The deployment (discussed more below) will likely use a cloud environment with DDoS protection. We can utilize rate limiting on both application (Express Rate Limit middleware) and infrastructure (cloud load balancer rules) to throttle abusive requests. WebSockets can also be limited by dropping connections from IPs that open too many sockets.

Logging and Analytics

Though not explicitly requested, for completeness: - We will implement logging on the server (using a library like Winston or Morgan for HTTP). Key events (logins, errors, admin actions, significant game events) will be logged with appropriate levels (info, warn, error). Logs can be shipped to a centralized system (like ELK stack or a cloud logging service) for monitoring. - Analytics: track metrics like daily active users, retention, common in-game events, using either a custom solution or an analytics service. This can help tune the game and also find cheaters (e.g. a user with an impossible progression curve stands out in analytics).

Frontend: Client-Side Implementation

The front-end will be delivered as a web application, running fully in the browser, with support for dynamic updates via WebSockets: - **Framework or Vanilla:** We have options to use a modern framework (React, Vue, Angular) to build the UI as a single-page app. This could ease state management and updates. However, since we also need to integrate an HTML5 Canvas and possibly we want to allow user-supplied CSS themes, a simpler approach might be to use plain JavaScript/jQuery or a lightweight library, combined with server-rendered HTML for some parts. Using a framework is not strictly necessary, but for maintainability, something like React with components for each UI panel could be beneficial. We could allow CSS overrides even with React by letting user load an override CSS file. - **HTML5 Canvas Rendering:** For visual components like an interactive galaxy map, battle viewer, or animations (like moving ships or an explosion effect when a building completes), we'll use Canvas or WebGL: - The Canvas can be used for a **galaxy view** (to plot stars and planets if we want a more visual map than OGame's tabular view). - In combat, perhaps visualize the fleets fighting in a simple 2D animation for fun (optional). - Any graphs or charts (maybe resource production over time) can be drawn on canvas or using SVG. - **User Interface Layout:** The UI will consist of: - A main navigation (Galaxy view, Empire overview, Buildings, Research, Shipyard, Fleet, Defense, Alliance, Messages, Shop, etc.). - Content panels that load respective information (we can use AJAX or WebSocket requests to fetch data for each panel). - We will make the UI **responsive** to some degree so it can be used on various screen sizes (though primarily desktop focus for such games). - Use CSS for styling and layout, making sure to define consistent class names and structure so that custom CSS can target them for recoloring or repositioning if needed. - **Applying Custom CSS:** Provide an interface in user settings to upload or paste custom CSS. The app can store this (maybe in the database associated with the user) and then when the user logs in, the client will load their CSS (e.g. insert a `<style>` tag or link to a generated CSS URL). We will have to regenerate or provide that CSS file perhaps by an Express endpoint (like `GET /user-theme.css?userId=...` that serves the saved CSS with correct content-type). This is an implementation detail but certainly doable. - **State Management:** The client will maintain some state like current selected planet, current building queue status (mirroring server state). On receiving events from server, it will update the relevant parts of UI. If using a framework, state management could be via

framework's system or even a Redux store if needed for global state. - **Error Handling and Offline Mode:** If the connection drops, the client should notify the user (e.g. "Reconnecting..."). If the server goes down for maintenance, show a message. We want a graceful handling rather than freezing. - **Asset Handling on Client:** We will have various images (icons for resources, images for buildings/ships, alliance logos, etc.). These will be packaged and served from a static directory or a CDN. We might create a sprite sheet or use modern web image formats for performance. Build tools like Webpack or Vite can help manage bundling of JS/CSS and asset pipeline (fingerprinting files for cache busting, etc.). - **Performance Optimizations:** Use caching where appropriate (e.g. the galaxy view or tech tree data can be cached client-side since it changes rarely). Use lazy loading (don't load all images at once, only when needed). - **Cross-browser compatibility:** Ensure the use of Canvas and modern JS features are tested on latest versions of Chrome, Firefox, Safari, Edge. We'll include necessary polyfills or transpilation for broad compatibility (target ES6+, since game players may use various browsers). - **Accessibility:** Not a primary focus, but we should ensure the UI is navigable and labels are clear. This also benefits general UX.

Deployment Strategy and Scalability

From day one, we aim to design the system to scale to a large number of concurrent players, as an MMO should: - **Server Deployment:** Containerize the Node.js application (Docker), making it easy to deploy on cloud services or Kubernetes. Use environment variables for configuration (DB URLs, secret keys, etc.). We might start with a single server process and scale vertically, but eventually horizontal scaling is needed for a truly large user base. - **Horizontal Scaling:** We can run multiple Node.js instances behind a load balancer. For WebSocket, we need sticky sessions or a consistent hashing scheme (to ensure subsequent socket upgrade requests go to the same server if not using a shared socket layer). A typical approach is using a Load Balancer with IP-hash or cookie to keep a user on the same server for their socket. With Socket.io + Redis, even if a user could hop, it wouldn't break because any server can reach them via Redis pubsub, but generally sticky is simpler. - **Clustering:** Even on a single machine, we can utilize multi-core by running a cluster of Node processes (using `cluster` module or a process manager like PM2). Each process runs an instance of the game server, and the cluster manager can distribute incoming connections. This yields better CPU utilization on multi-core servers. - **Database Scaling:** - *MongoDB:* Deploy as a replica set for failover. For scaling writes/reads, consider sharding by player ID or universe if needed. Most queries (one player's data at a time) might scale well vertically, but things like ranking all players is more intensive (we mitigate that with Redis). - *Redis:* Use a managed Redis or Redis cluster if needed, but often one Redis instance (with proper memory and possibly replication for failover) can handle enormous throughputs given its in-memory nature. - **Stateless vs Stateful:** The server will maintain some transient state in memory (like the event queue, currently logged-in users, etc.), which means if a server instance goes down, those events need to be recovered by a standby or on restart. We will rely on the DB to persist important events, and possibly have a watchdog that reassigns events. For example, if we have multiple servers each handling a subset of galaxies, we'd need to redistribute on failure. Initially, we might keep it simpler: any server can handle any event, and all watch the DB for events due. But more realistically, we could partition by something (like galaxy number) to reduce contention. - **Microservices consideration:** Initially, we implement as one application. But we keep in mind possible separation: - A separate **combat resolution service** could be made if that becomes a bottleneck (microservice that takes combat input and returns outcome, could scale independently). - A separate service for handling payments (for security and isolation). - A separate chat service (though Socket.io can handle chat easily integrated). However, unless needed, we avoid splitting too early to reduce complexity. - **CDN for Static Assets:** Offload images and large JS bundles to a CDN for faster load globally. The game's static files can be deployed to something like AWS S3 + CloudFront or similar. - **Continuous Deployment:** We will set up a CI/CD pipeline that runs tests, then

deploys to a staging environment. For production, we might do rolling deployments to avoid downtime (containers or processes spun up gradually). - **Scalability Testing:** We will perform load tests to gauge how many concurrent users one server can support (with typical action frequency). If each Node instance can handle e.g. 2000 concurrent sockets with acceptable latency, and we expect 20,000 players, we'd deploy 10 instances. We also ensure Mongo can handle the write load (we might need to tune write concern or use batching for frequent small updates like resource ticks). - **High Availability:** Use health checks and auto-restart for Node processes (PM2 or Docker restart policies). Set up monitoring for server metrics and game-specific metrics (like queue lengths, event delays). If a server goes down, the load balancer will route to others, and the game should continue seamlessly for users (they might experience a brief reconnect). - **Backups and Recovery:** Regular backups of databases and perhaps snapshots of server state (though state is mostly in DB). If something catastrophic happens, have a procedure to restore data with minimal rollback (maybe daily backups for game state since continuous is tough for an MMO but at least something). - **Geographic Scaling:** If the player base is worldwide, consider deploying servers in multiple regions (e.g. one in Europe, one in Americas, etc.) each hosting separate universe instances to minimize latency for regional players. Real-time games benefit from servers close to players. If one unified universe is desired, we'd keep it in one region to avoid inconsistent latencies (since cross-region real-time is tricky). - **Deployment Environment:** Could use cloud VM (AWS EC2, etc.), or use orchestrators like Kubernetes for ease of scaling and self-healing. Using Kubernetes, we can define deployments for the Node server, MongoDB (or use a hosted Mongo service like MongoDB Atlas), and Redis (or a hosted Redis). This also simplifies scaling: just increase replicas for Node server, and if stateless aside from DB, it scales out.

Development Roadmap

To build this project, we will proceed in iterative phases, each delivering a functional subset of the game and then expanding on it. Below is a high-level development plan:

1. Project Setup & Foundation (Phase 1):

2. Set up the development environment with Node.js and Express. Initialize the project structure (using TypeScript if chosen for type safety).
3. Configure MongoDB and Redis connections. Define the initial schema models for Player and Planet.
4. Implement **user registration and login** (API endpoints, database persistence, and basic JWT auth or session handling). Verify that a user can create an account and log in.
5. Set up the basic Express server routing and serve a simple homepage to ensure deployment pipeline works early.
6. Include a basic Socket.io setup: e.g. when a user logs in, establish a socket connection and echo a test message to verify real-time channel.

7. Core Game Mechanics (Phase 2):

8. **Planet Management:** Implement planet creation for new players (on registration assign a homeworld). Build out the planet data model and retrieval of player's planets. Create a simple UI page that lists the player's planet and resources.
9. **Resource Production:** Implement the resource generation logic on the server. For now, could simply accrue resources over time or on request. Display current resource amounts on the UI and update them periodically (perhaps using a socket event every few seconds to push new totals).

10. **Building Construction:** Define a list of building types and their cost/time formulas. Implement constructing/upgrading a building:
 - API or socket event to start construction (with validation of requirements and resources).
 - Schedule the completion (store finish time). For initial development, you might speed up time or allow instant completion for testing.
 - On completion (use a simple setTimeout in this phase), update the level and notify the client.
 - Front-end: a Buildings UI that shows current levels and a button to upgrade (disabled if requirements or resources not met). Show a countdown for building in progress.
11. **Research Tech Tree:** Similar to buildings – implement a few research items and allow researching one at a time. Enforce that only one research at a time globally (for the player) and apply research effects (like bonus production or unlock conditions).
12. **Fleet Basics:** Implement the Shipyard building and allow constructing ships (which consumes resources and takes time, similar to buildings). Maintain counts of ships at a planet.
 - No missions yet, but ensure you can build a fleet.
13. At the end of Phase 2, a player should be able to: register, log in, see their planet, build mines and other structures, research basic tech, and build some ships. Essentially the economic engine is in place. This phase includes a lot of server logic and verifying it with unit tests (e.g. test that resource production formula yields correct amounts, test that you cannot build without cost, etc.). Also, by this phase, set up the admin account manually in DB to allow future admin operations easily for testing.
14. **Combat and Fleet Missions (Phase 3):**
15. **Galaxy and Navigation:** Create a representation of the universe (galaxies, systems, coordinates). Implement a Galaxy View UI where players can see other planets (initially just their own if alone, but later other players). This requires seeding the universe – possibly generate coordinates and maybe allow one or two test players for combat.
16. **Fleet Dispatch:** Implement sending fleets on missions:
 - Create the UI form to select ships and target coordinates and choose a mission type.
 - On server, handle the fleet dispatch: validate enough ships are available, remove them from the planet (assign to a moving fleet), calculate arrival time.
 - Store the fleet in transit (in DB and memory schedule).
 - Emit an update to the user showing the fleet mission en route (with arrival countdown).
17. **Combat Simulation:** Implement the battle logic on arrival:
 - For testing, perhaps implement a simplified version first (one round resolution or just compare power) to ensure the flow works, then refine to the full multi-round simulation.
 - If the target is another player's planet, retrieve defending ships/defenses, run the combat, determine outcome.
 - Update both players' states (deduct ships, create debris, loot resources).
 - Create a combat report and send to both players (maybe as a simple text for now).
18. **Other Missions:** Implement transport (which simply moves resources), deploy (move ships to another owned planet), and return logic. Ensure fleets can be recalled mid-flight (optional but OGame allows recall on deployment missions, ACS, etc.).
19. **Debris and Recycling:** After combat, allow Recycler ships to collect debris. Implement the mission logic for recycling.

20. **Alliance Basic (if time in this phase):** At least implement alliance creation and joining, so that we can test ACS (alliance combat) in the next phase. Possibly skip detailed alliance features until next phase, but have the data model ready.
21. At the end of Phase 3, the game is essentially playable in a basic form – players can attack each other and the core loop of build > gather > fight is there. We will do internal testing with a few players to balance combat outcomes and ensure there are no major bugs in mission scheduling.
22. **Social and Advanced Features (Phase 4):**
23. **Alliance System:** Complete the implementation of alliances:
- UI to create alliance, invite or apply, view member list.
 - Alliance chat or at least an alliance message board.
 - Implement ACS fully: allow allies to ACS defend or attack. This might involve an interface to invite allies to a mission and coordinating arrival times.
 - Alliance management features for leaders (kick, change roles).
24. **Ranking and Leaderboards:** Calculate player scores based on their assets. Implement the leaderboard page and the backend logic to update scores on relevant events. Use Redis for efficiency as planned. This phase may involve a background job or cron that recalculates score periodically (say every hour) if not doing instant updates.
25. **In-Game Messaging:** Develop the messaging system for players (send message, inbox, outbox). Ensure these are stored and retrievable. Also include system messages (combat reports are basically messages). This increases player engagement and is essential for coordination outside of alliances.
26. **In-Game Shop:** Open the shop interface:
- List a few premium items (for testing, maybe give some free premium currency to test buying).
 - Integrate a test payment flow or at least an admin grant of premium currency to simulate purchases.
 - Make sure using a premium item (e.g. buying a resource booster) correctly affects the game state (for instance, if a booster reduces construction time by 50% for 1 hour, the server should check for that when calculating times).
27. **Advertising Hooks:** Integrate placeholders for ads. For development, we might not include actual ad network scripts (since those require production domains), but structure the UI to accommodate an ad banner and test with a dummy image or content. In final deployment, we'll insert the actual ad code.
28. **Polish Combat & Fleet UI:** Enhance the combat report format (make it more readable, possibly include animations in a Canvas-based battle viewer if we want extra flair). Also provide better fleet management UI (like showing all fleets in flight with their details, and maybe an option to cancel if applicable).
29. **Notifications:** Implement visual/audio notifications for important events (attack alarms, build complete alerts). Possibly even email notifications for being attacked (could be a stretch goal, but OGame had these features).
30. **Testing:** At this phase, we should do extensive playtesting and gather feedback. We'll also perform security testing (try to break the game as a fake cheater, ensure the server rejects invalid operations). Fix any balancing issues discovered (tweak production rates, combat formulas if needed).
31. Now the game should be feature-complete with alliances, warfare, economy, and monetization.

32. Administration & Post-launch (Phase 5):

33. Build out the **Admin Panel** with web pages for admins to log in and perform actions:
 - Start with read-only views: see list of players, search player by name.
 - Add controls: ban/unban, edit player resources, etc., as identified in spec.
 - Moderation tools: view chat logs, possibly implement a simple keyword filter in chat to demonstrate moderation capabilities.
34. **Analytics & Logs:** Set up any analytics events (like track new registrations, daily active count). This might involve integrating a third-party analytics or simply logging and analyzing manually.
35. **Scaling and Deployment Prep:** Before launch, do load testing (simulate many concurrent users performing actions) to identify bottlenecks. Optimize any slow queries or server functions identified.
 - E.g. if combat resolution is slow, consider optimizing the loop or distributing the load.
36. Prepare the production environment:
 - Set up Docker containers if using them.
 - Configure domain, SSL certificates.
 - Launch the database servers (perhaps use a managed MongoDB/Redis service for reliability).
 - Use a process manager (PM2 or Kubernetes) to run the Node app and ensure it restarts on crash.
37. **Soft Launch/Beta:** Launch an open beta with a small group of users to observe stability. Use this to tweak balancing and fix any issues that only appear with real user behavior.
38. **Official Launch:** Scale up servers as needed and onboard more players. Continue to use the admin tools to run events (maybe special universes, etc.) and monitor health.

Throughout all phases, we will maintain a high standard of documentation and utilize version control (git) for code. Each feature will be accompanied by tests where feasible (especially for game logic like combat calculation and economy, which are easily unit-testable). Code reviews will ensure quality and security.

By following this development plan, we will gradually build a complex multiplayer game in manageable increments, ensuring at each step that we have a working product (even if minimal at first) and avoiding big bang integration issues. The end result will be a **fully functional browser-based MMO** with the depth of OGame and a modern tech stack, ready to engage players.

Conclusion

In summary, this project entails recreating the rich feature set of *OGame* – real-time strategy, empire management, and multiplayer combat – using a modern JavaScript stack. We will utilize **Node.js + Express** for a fast, modular backend, with **WebSocket** real-time updates to provide a smooth multiplayer experience. Data will be stored in a combination of **MongoDB** (for flexible, scalable game data storage) and **Redis** (for caching and real-time pub/sub tasks) to ensure performance at scale ¹. We'll implement all core mechanics: planetary economies, tech trees, fleets and battles (with turn-based simulation up to 6 rounds), as well as alliances enabling group strategies ³. A robust admin panel and careful security measures (authoritative server, input validation, anti-cheat checks) will keep the game fair and stable. The front-end will leverage HTML5 Canvas and user-customizable CSS to deliver an enjoyable and personalized experience to each player. With a clear development roadmap guiding us from foundational systems to advanced features, this browser-based multiplayer RPG will be built for both **depth** and **scalability**, ready to welcome a universe of commanders to vie for galactic supremacy.

Sources:

- OGame Wiki – game overview and mechanics ³
 - Technical discussions on Node.js game servers, real-time networking, and databases ¹
 - Express.js Official Documentation for backend framework selection.
 - MongoDB vs SQL for game data – GameDev StackExchange.
 - Tencent Cloud Game Database recommendations (Redis for real-time, MongoDB for scaling) ¹.
 - StackOverflow – WebSockets for multiplayer games (WebSockets offer lowest latency for browser real-time games).
 - OGame resource and premium currency info.
 - Reddit GameDev discussions on Node.js game loops and Socket.io usage.
-

¹ ⁴ Any recommendations for game databases? - Tencent Cloud

<https://www.tencentcloud.com/techpedia/129504>

² ³ Combat | OGame Wiki | Fandom

<https://ogame.fandom.com/wiki/Combat>